

Implementing from SDL

RICHARD SANDERS



Richard Sanders (41) is Research Scientist at SINTEF Telecom and Informatics, and lecturer at the Norwegian University of Science and Technology (NTNU), from which he graduated in 1984. He has previously worked as a consultant with CAP Gemini and as a software designer and manager at Stento, developing intercom systems and participating in the SISU project. He joined SINTEF in 1995, and helps introduce new development methods in companies, holds courses and carries out research in the field of system development methodologies. He is currently undertaking a PhD at the Institute of Telematics at NTNU.

richard.sanders
@informatics.sintef.no

SDL is rated as a fine modelling language by researchers and industry alike. But can it be used to derive efficient implementations? This article sheds light on some of the issues involved.

1 Introduction

One of the positioning features of SDL, and possibly the main reason for its adoption by an increasing number of users outside the traditional telecom domain, is the ease in which SDL models can be transformed to a wide range of popular programming languages. From the early days where such transformations were done manually, to the present situation with good tool support for automated transformations, the transformation from a formal model in SDL to an actual system running on a computer has become relatively short and well paved.

Newcomers to the SDL world quickly adapt to the state-of-the-art of the SDL CASE tools that are available. Given good tools users can experience the true benefits of design-oriented development. These benefits include an executable model of a system, enabling simulation at different levels of granularity, and intuitive traces of the system's actual or intended behaviour expressed in Message Sequence Charts. Advanced model checking facilities are available, and a fairly straightforward means of generating implementations both for host and target. All this can be achieved while maintaining much of the tool functionality even in the target implementation environment.

In the earlier days of SDL, hand coding from specifications (e.g. protocol standards) and system designs was commonplace. Engineers found practical solutions to the manual transformation from SDL to running systems, and reported good results, such as fewer errors and easier maintenance. Despite the manual work involved, the implementation process was efficient, and typically counted for less than 30 % of the total development effort, testing included. Many products were developed this way, and resulted in an increase in the number of SDL users. Since the end of the 1980s this manual coding scheme has increasingly been replaced with automatic generation of code by tools.

Design orientation

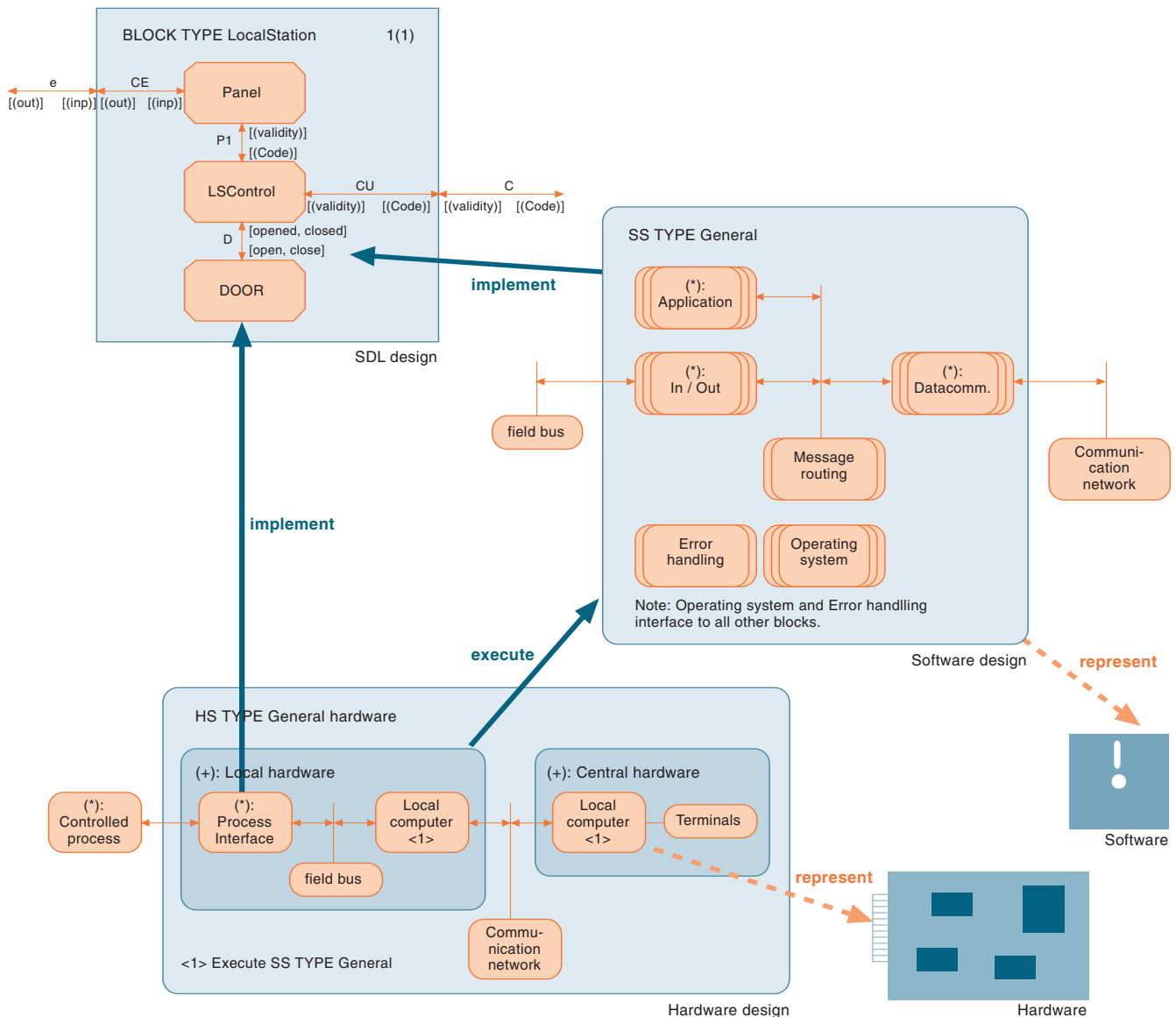
Design orientation is characterised by a development process where systems are understood and maintained mainly in terms of abstract design descriptions, e.g. in SDL and MSC. This is considered to be a more mature process than *Implementation orientation*, where the actual programming code is the only true description of the system, and any design models that exist act as illustrations only. The step up from Design orientation is *Property orientation*, where the focus is on desired properties.

SDL encourages the translational approach¹⁾ to design, as opposed to the elaborational approach favoured by methods like Booch and OMT, and which to a certain degree is still true of many UML methods. The benefits of a translational approach with automatic code generation are obvious and imminent:

- The SDL system and the implementation are by nature consistent, which was often not the case with manual coding from informal SDL;
- Systems can to a large extent be maintained solely on the SDL level, keeping complicated behaviour expressed in a form more easily understood by humans (i.e. MSC and SDL), in contrast to machine-friendly programming languages (e.g. C++, Java, Chill, etc.).

In this article we shall dwell slightly on these translation issues, that is, how the SDL primitives can be turned into software running on hardware. Newcomers to SDL and automated program generation often consider such issues uninteresting, trusting tool support to cater for the technicalities involved. And while the latter attitude generally speaking is viable, some words about the transformation from the ideal

¹⁾ The translational approach involves transformation between different models at different levels of detail, e.g. from an implementation-independent model in SDL to code in C++. This in contrast to the elaborational approach, which is distinguished by re-working and adding details into models as you progress down the development life cycle, until the model is to some extent complete.



world of formal definition techniques to the accidental world of programming languages and computers can be worthwhile, since many implementation choices are available. Indeed, knowing and controlling them is crucial for anyone needing to remove bottlenecks and optimise performance. This activity is here called Implementation Design.

2 The Ideal World of SDL

SDL is a language with an ideal view of the world. It is based on the concept of finite state machines communicating via asynchronous³⁾ messages. The underlying SDL machine of each process contains a single FIFO queue of infinite

length where all signals to the process arrive. SDL processes consume one signal at a time from the FIFO queue, and perform a transition on that signal. Transitions take an “insignificant” amount of time, and run to completion once they are started. Note that processes operate in parallel, so the transitions of many processes may run at the same time. All data belong to processes⁴⁾, and are described by abstract data types of possibly infinite range. SDL channels are secure; SDL signals never disappear in a signal transmission.

Additional SDL constructs include:

Figure 1 Implementation descriptions illustrate the coupling between functional design and implementation design²⁾

²⁾ This figure uses the SOON notation [1] to depict Software design and Hardware design. See also [4, 8, 9, 10]. See [11] to see to what extent UML stereotypes can model the same as SOON.

³⁾ A kind of synchronous communication is supported in remote procedure calls.

⁴⁾ SDL 2000 now allows data to belong to blocks, though access to the data is controlled by the state machine associated with the block.

Implementation design

The goal of Implementation Design is to define the mapping between the abstract system defined in SDL, and a concrete system made up of hardware and software components. [1]

- Process identity values (Pid) used to identify communicating peers (no specific implementation is presumed by SDL);
- Priority input;
- Enabling conditions;
- Remote procedure calls;
- A save queue for signals to be treated after the next change of state;
- Importing and exporting variables from/to other process;
- Timers that can be started or stopped, and that signal timeouts in the form of signals in the FIFO queue. SDL recognises time ticks, but does not define the interval between two ticks.

For space and simplicity, these and other SDL mechanisms are not elaborated any further in this article. An overview of the interior of an SDL state machine is shown in Figure 2, see also the paper by Rick Reed on SDL-2000 for new millennium systems in this issue.

3 The Not-So-Ideal Real World

SDL is different from the real world in many respects. Real systems have finite resources like limited queue lengths and maximum value ranges for data variables, and programs on a single computer do not run truly parallel. Real systems are subject to errors, e.g. the malfunctioning of hardware, and the corruption of sig-

nals on unreliable communication paths. In addition there may be different synchronisation needs between a system and its environment than the asynchronous, time-independent world of SDL messages.

There is a design gap between SDL and the world of programming languages [1]:

- **Concurrency.** Sequential programming languages like C or Java give no support to the concurrency of SDL. Some languages like CHILL and ADA support concurrency but do so differently from SDL.
- **Timing.** Very few programming languages support time at all. SDL-like timing is not directly supported by any language, not even by CHILL.
- **Communication.** SDL-like signal communication is not directly supported by any programming language.
- **Sequential behaviour.** An SDL process graph specifies state-transition behaviour in the fashion of an extended finite state machine. Programming languages specify action sequences.
- **Data.** SDL data are abstract and possibly infinite. The implementation in a programming language has to be concrete and finite.

But many of these differences can be overcome with affordable measures. Some of these measures and techniques will be introduced in the following sections.

3.1 Implementing Signal Transfer

Message transfer in the ideal SDL world can most generally be implemented using an asynchronous messaging technique, or in special

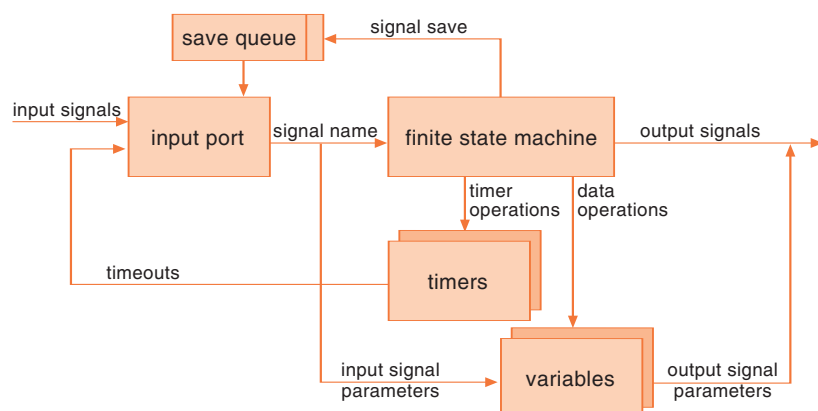


Figure 2 The SDL machine

cases as continuous values, or procedure calls or method calls in the programming language.

3.1.1 Signals Implemented as Method Calls

Using method or procedure calls, the signal sending is implemented by calling a method implemented in the receiving process. That method contains the transition of the receiving process. This synchronous technique is the most time efficient, but can only be used in special cases where the following conditions are satisfied [1]:

- The SDL behaviour must have the nature of a procedure call tree (e.g. A calls C who calls E, who returns to C, who calls F, who returns to C, who returns to A; see Figure 3).
- Only the SDL process at the root of the tree can receive signals from the environment or run SDL timers (process P1 in the example).
- Any response modelled as SDL signals must be implemented as return values in the procedure call, which limits the number of return signals to exactly one, which must be the last SDL action in the transition of the called process.
- The maximum processing time for any such call tree should not exceed the arriving rate of external signals to the root process.

This implementation scheme⁵⁾ is time and space efficient in software, since it relies on some of the most basic primitives of programming languages and processors (e.g. using stack registers). The scheme is a common technique used for message calls in traditional designs of information systems modelled in e.g. OMT or UML. It implies synchronous communication semantics, and has inherent limitations that make it inappropriate for communication structures that form a network. The scheme is only viable when the mentioned set of criteria is fulfilled. These criteria are seldom present in reactive real-time systems, hence schemes involving asynchronous message communication and flexible scheduling are used.

The method call scheme can be used with advantage in parts of an SDL design where these criteria are satisfied, while other parts use the more general implementation technique of buffered messaging mentioned below. Optimisation performed during manual coding typically involved

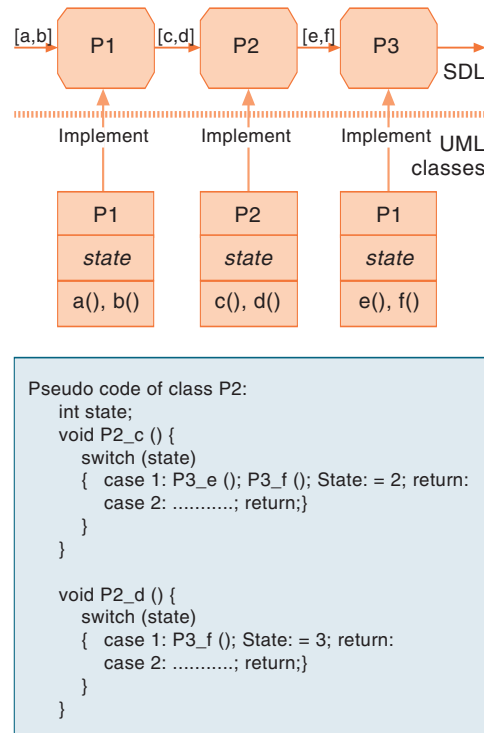


Figure 3 Implementing communication with method calls

changing the SDL model so that these criteria were met within components faced with high performance requirements. Commercial tools for automatic code generation do not seem to address this issue very well (although the criteria analysis could well be performed by the tools).

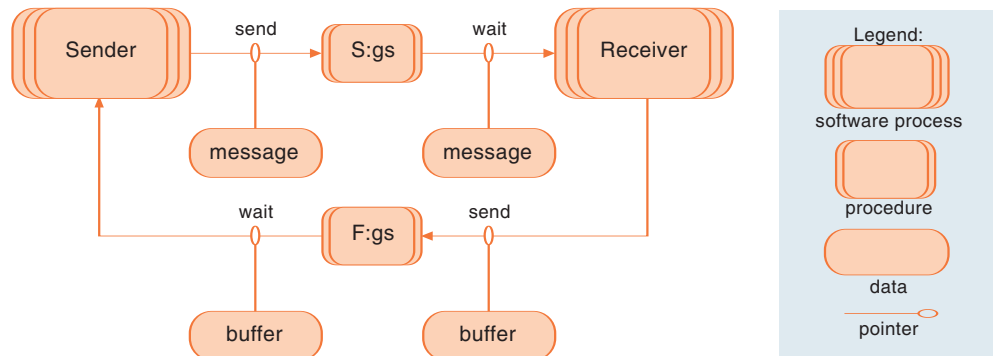
3.1.2 Signals Implemented by Message Buffers

As stated above, the most general implementation of message sending involves buffered communication. This separates communication from activation, but requires additional memory (the buffers) and processing (buffer handling) compared to procedure calls. A very general approach involving parallel software processes (here denoted *Sender* and *Receiver*) that communicate using a general semaphore *S* is shown in Figure 4 [1].

The infinite message queues of SDL do not exist in the real world, but by employing techniques like semaphores providing back pressure, or exception handling at buffer overflow, the sending processes may be temporarily suspended. Then the receiving process can consume a signal from the limited sized input buffer, thus making room for additional messages. Protection using semaphores is shown in Figure 4, where *Sender* must fetch a free buffer from the freepool *F* prior to

⁵⁾ Note that this technique departs from the SDL semantics of run-to-completion for transitions that applies between SDL processes.

Figure 4 Concurrent buffered communication



sending a message to *Receiver* via *S*. *Receiver* will return the buffer to the freepool *F* after consuming the message. *S* and *F* are general semaphores containing queues, which are typically well supported by operating system kernels.

3.1.3 Optimising Parameter Passing

A final remark on the implementation of signal transfer is concerned with the parameters of SDL signals. In SDL, a process can only get hold of the input parameters by copying them into process variables. Likewise, parameters have to be explicitly set in output signals. There is no built-in mechanism to forward a received signal to some other receiver, complete with parameters.

This typically leads to a lot of copying in and out of parameters, and large amounts of process data required to hold values that are of no interest to the process as such. In designs of protocol stacks, this is particularly apparent. Many newcomers to SDL react to this, and view it as a waste of time and space.

However, this need not be the case in the implementation. An analysis of the SDL can show that parameters are e.g. forwarded largely unchanged, and an implementation can e.g. reuse the message buffer without returning it to the freepool in Figure 4. Or analysis can reveal that process data are only used to hold values for the duration of a transition, and never longer. In an implementation, such process data need not be stored in non-volatile memory, but instead placed on the stack. These are examples of optimisations that are common-place in good compilers, and should be found for SDL compilers too. See also [2].

Note, though, that there are limits to what a general code generator can analyse; if a process calls external code, the code generator may not be able to perform a complete analysis. If so, other mechanisms must be used, e.g. special annotation or commenting of signals and data.

3.2 Implementing SDL Processes

The implementation scheme most close to the ideal SDL world is realising each SDL process

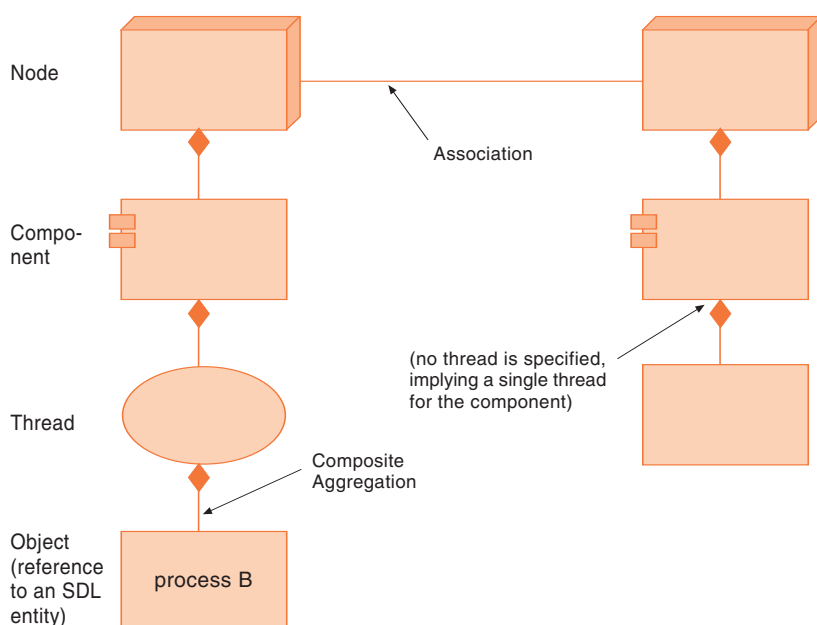


Figure 5 UML deployment diagrams used to express software structure (from Telelogic deployment editor)

instance on its own hardware processor. Thus every process runs truly parallel. This is seldom feasible, and the most common technique is to collect a number of SDL process instances in one software process (also known as operating system process or task), and have a number of such software processes running at different levels of priority.

SDL processes are typically allocated to high priority tasks (like timing and I/O processes), medium priority tasks (the application processes), and low priority tasks (SDL processes that perform background jobs performing statistics, monitoring, logging and error reporting).

This type of software design decision is outside the scope of SDL, and cannot be formally captured in the SDL model. Instead, additional descriptions are necessary, see Figure 1. Design methodologies like TIME [3, 16] treat implementation design as a topic of its own, and SDL tools typically have additional functionality to help such grouping and code distribution in the program generation process. An example is the deployment editor in Telelogic TAU, see Figure 5. See also the paper by Philippe Leblanc, Anders Ek and Thomas Hjelm on Telelogic SDL and MSC Tool Families in this issue.

3.3 Implementing State Transition Diagrams

The term *process* has two distinct meanings; an *SDL process* is a Finite State Machine, while a *software process* (also known as operating system process or task) is a unit of scheduling and priority in a running software system. One of the important decisions during Implementation design is deciding the mapping from SDL process to software process; this can be decisive for the properties of the resulting system.

The state diagrams of SDL that model the actual behaviour of the SDL process can be implemented in a number of ways, with various results regarding execution time, space required, and support for range of SDL constructs, see [1].

The *state-oriented method* uses the CPU's program counter to store the state of the SDL process, with GOTO jumps for state changes. On the one hand, this scheme saves time and space in the implemented software process, but on the other hand wastes time and space by requiring more software processes, since it limits the mapping to a 1:1 relationship between SDL process instances and software process instances.

The most general scheme is to use a *table form of the state diagram* combined with common programming language constructs. The current state of each process is typically stored in a process variable along with the process data, and the state diagrams are stored in table rows for fast look-up at transition time. Multiple instances of a single process type or of different process types can be grouped together to run in the same software process, providing a N:1 mapping.

3.4 Implementing Abstract Data Types – and Adding Legacy Code

Not all detailed working of a system deserves to be visible at the SDL level. Examples include aspects like device drivers, complex algorithms and complex operations on data. The SDL way of concealing some of this detail is by extended use of operations on user defined data types. Data types are defined on the SDL level, typically by building upon the predefined SDL data types, or by making new types. New data types are defined from the predefined types, e.g. by restricting the value range, thereby providing a mapping to programming language constructs.

syntype BYTE integer constants (0:255);

For example, the ideally infinite range *integer* of SDL can be used to make an 8-bit *BYTE* type:

The *BYTE* variables can then easily be mapped to e.g. "short unsigned int" in C, for a given compiler for a given processor.

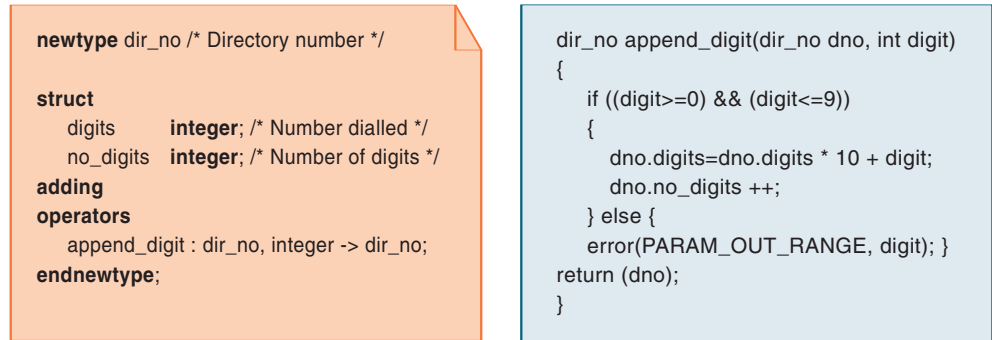
SDL-96 [4] allows operations on data to be described with axioms. Axioms are not constructive, and are seldom used in practice⁶⁾. Instead, operations are defined by their signature at the SDL level, and are implemented in the programming languages of your choice. An example of this is given in Figure 6.

Hand coding SDL operations is one method of interfacing manually written or legacy code into the SDL world. Another way of interfacing legacy code is by concealing a legacy part in dedicated SDL processes, e.g. a database system or a GUI. SDL tool vendors provide good support for such code integration, although reverse engineering is not yet generally available from commercial CASE tools⁷⁾ (many contend that reverse engineering is not applicable to SDL systems engineering). One of the benefits of

⁶⁾ Axioms were removed from the SDL-2000 standard [9].

⁷⁾ Work within reverse engineering to SDL is currently in progress, e.g. at the Software Engineering Institute at Moscow [12].

Figure 6 Abstract data type in SDL-96 with corresponding C code for an operator



interfacing legacy code in dedicated SDL processes, is that this maintains a loose coupling between the SDL part and the legacy part.

3.5 SDL Runtime Support

A common technique employed when implementing SDL systems is introducing a layer of software called the SDL runtime support between the application processes and the operating system that runs on the hardware. Novice SDL users may not be aware of this layer, since program generation tools typically offer code generation to both naked processors and a range of operating system kernels like PSOS, VRTX, etc. The SDL runtime support implements many concepts of the SDL machine (see Figure 2),

with some limitations due to the nature of the real world, thereby making the task of implementing the actual SDL processes much simpler, see Figure 7 [1].

4 Support for Program Generation from CASE Tools

Generally speaking, the commercially available tools give good support for program generation, and the common SDL application developer does not need to spend much time analysing what schemes are used in the program generation. The dedicated application developer probably never needs to read the generated code, which may be of interest during initial integration testing only. The resulting programming

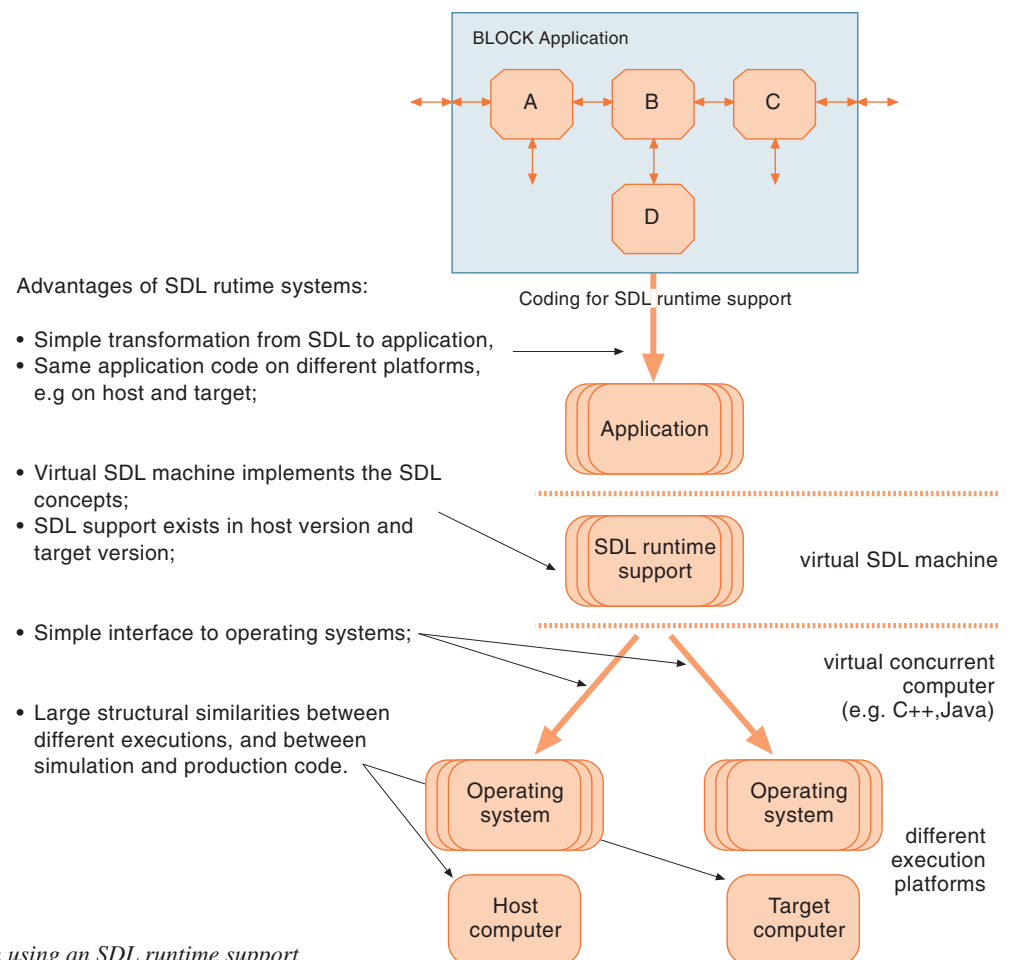
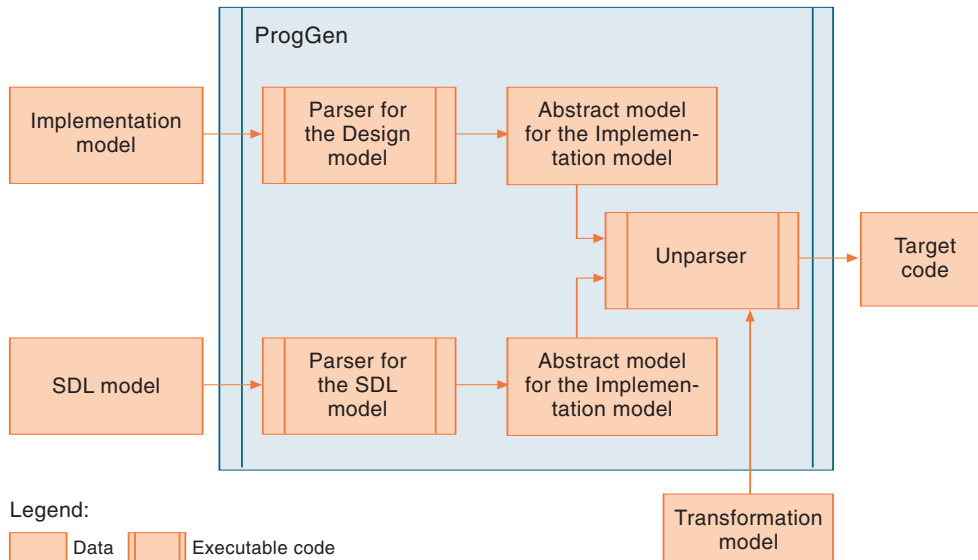


Figure 7 Layered implementation using an SDL runtime support

Figure 8 ProgGen's architecture



code can in general be untouched by human hand – and unread by human eye.

But the same does not apply to the necessary few who are responsible for mapping the ideal design onto the real world. And this is not always a trivial task, as this article hints at. To get optimised code for a new product, blood, sweat and tears may still be required. And although tool vendors have and are investing considerable effort in facilitating the program generation process (a fact that is reflected in the considerable rise in price of the toolkit when program generation is added), there is still much left to be desired.

It is not yet the case that the implementation designer can choose different implementation techniques for different SDL components in a system, e.g. to optimise for space in one instance, and size in another. Or that implementation speed and size are automatically adjusted depending on which SDL constructs are used in an actual design.

SINTEF has been working with program generation for over 15 years, and has developed a very flexible program generator named ProgGen [5, 6]. ProgGen takes as input the textual version of an SDL design, and outputs code in a variety of formats and programming languages, depending on the skeleton files defined by the implementation design. This tool has recently been extended with the capability of selecting different mapping strategies for different parts of an SDL

design, according to implementation design decisions fed into the program generator in a textual form [7], see Figure 8. Although ProgGen is not a fully-fledged tool (e.g. it currently only supports SDL-88 and SDL-92 [8]), it shows what can be obtained from using flexible tools.

5 Consequences of Program Generation to the Development Process

Working in a design-oriented paradigm, users report higher quality and higher productivity, especially when maintaining or enhancing systems designed using SDL. Many companies have over a decade of experience with automatic code generation from SDL, and are constantly increasing the penetration of SDL in their reactive real-time system designs.

Industry typically reports that errors delivered to the market are reduced by at least 50 % compared to a pre-SDL development process, and that an increased productivity of 20–30 % in software design has been measured⁸⁾. Even higher yields in the area of test generation and execution can be expected when TTCN⁹⁾ is introduced. The added cost of investing in a new development process is typically earned within the first project, with subsequent projects reaping the full benefits. Developers can more easily move between projects, projects become less person-dependent, and project progress is easier to define and monitor (less of the “90 % finished” syndrome). This is good news for project leaders, management and designers alike.

⁸⁾ Data from the Norwegian SISU project in 1989 – 1996 [13]. CASE vendors boast of even higher earnings [14].

⁹⁾ Test suites in Tree and Tabular Combined Notation (TTCN) can be semi-automatically derived from SDL and MSC models [15].

Figure 9 Example of inline code in an SDL process graph

```
'inline code'
/*#CODE
jobdataptr->int1 =
(JOBDT*) init (
&(pr_ext->param1));
*/
```

Some words of caution should be mentioned, though. One direct consequence of automatic code generation is that SDL is used much as a graphical programming language, substituting textual languages such as C++ and Java¹⁰⁾. SDL was initially not designed to be a programming language, and has until SDL-2000 left much to be desired in that respect. Many constructs expected by programmers have been absent or were cumbersome, e.g. expressing behavioural loops¹¹⁾.

The CASE tools supporting inline-programming directives in native programming languages have introduced another drawback. It is easy to understand the motivation for inline code, since it simplifies the task of generating complete applications in some chosen programming language. But it defies the objective of building implementation-independent models; one can no longer change implementation language without going through the whole model and altering the inline code.

One aspect to be kept in mind by SDL designers wanting to generate code automatically, is the need to be formal. If one has the habit of using SDL slightly informally, and wants to start generating code, the need to be precise and detailed becomes apparent. Loose ends must be tied up, and informal use of SDL constructs revised. These revisions also apply when simulation and formal analysis of SDL system is performed, so this extra work is not a weighty argument against code generation, since simulation gives a high return on investment. There are reports from industry that an additional 30 % effort is required to fully formalise SDL designs, compared to designs in informal SDL. This increase

in design effort, though, is quickly earned in the later maintenance and enhancement phases.

Note that some SDL tools support the simulation of informal SDL, a feature that is attractive for high level designs.

Some companies evaluating automated code generation have become aware of the penalty involved in the way of memory requirements and processing overhead introduced by some of the implementation platforms. Although these requirements vary depending on e.g. the level of debugging requested, and the fact that different target operating environments are supported by CASE tools¹²⁾, one can still find examples of manual coding that are more efficient.

With manual coding you can take advantage of your knowledge of the environment, using shortcuts to save time and space. This can work fine, as long as the assumptions are valid and are understood. One can single out particularly demanding parts to be hand coded or realised in hardware, while other parts are generated automatically.

There is a strong pressure on tool vendors to improve the code optimisations, like there used to be when 3rd generation programming languages were introduced. Programmers now generally tend to rely on the quality of compilers, and do not write assembly code by hand unless they really have to. Hopefully, the same will soon apply for SDL code generation, if it does not apply already.

6 Impact of SDL 2000

Many of the new features added in the 2000 version of SDL [9] have taken into account the fact that SDL has become popular as a graphical programming language. For instance, it is now possible to introduce temporary data in transitions, that easily can be implemented as stack variables, thereby limiting the static memory requirements of an SDL process implementation; only data with a life span longer than a transition needs to be stored in persistent memory.

Exception handling, similar to what you find in C++ and Java (and CORBA), is defined in SDL

¹⁰⁾ Interestingly enough, designers of digital hardware have moved in the opposite direction. Textual hardware description languages like VHDL and Verilog substitute the earlier graphical hardware expressions of the schematics and block diagrams. In general the goal is to strike a good balance between pictures and words. SDL is possibly close to this goal within its realm of description.

¹¹⁾ Many of the changes to SDL introduced in SDL 2000 were motivated by the current use of SDL as a programming language, and has resulted in many requested programming features, like loop constructs and stack variables (see later).

¹²⁾ E.g. C-micro from Telelogic.

2000. This is depicted in special graphics in the process graphs.

The action language introduced in SDL 2000 supports typical programming constructs such as loops and conditions, features which “SDL programmers” have missed. This results in a textual representation of what previously was space-consuming graphics, or was hidden in operators or, worse yet, as inline code in some programming language, see Figure 10. The action language should remove the need for much of the inline code that is all too often found in SDL designs, and should bring SDL design back toward the implementation-independent track where it originally was and should be.

We conclude that SDL is moving in a direction that is favourable for those wanting to derive their code from SDL, and who wish to maintain systems at the SDL level. It is possible to combine designs in SDL with models in UML that cover other system aspects (like database systems and GUI). Tool vendors are working hard to implement the aspects of SDL 2000 most needed by SDL users, especially in the field of program generation. This, along with the other aspects of SDL treated in other articles in this issue, will continue to increase the adoption of SDL in product development world-wide.

7 References

- 1 Bræk, R, Haugen, Ø. *Engineering Real Time Systems*. Hemel Hempstead, Prentice Hall International, 1993.
- 2 Henke, R, König, H, Andreas Mitschele-Thiel, A. Derivation of Efficient Implementations from SDL Specifications employing Data Referencing, Integrated Packet Framing and Activity Threads. In: *SDL'97. Time for Testing, Proceedings of the Eighth SDL Forum*, 397–414. Amsterdam, Elsevier Science, 1997.
- 3 Bræk, R et al. *TIME : The Integrated Method*. 2000, October 12 [online]. – URL: www.sintef.no/time.
- 4 ITU-T. *CCITT Specification and Description Language*. Geneva, ITU-T, 1996. (Recommendation Z.100 (SDL'96).)
- 5 Floch, J. Supporting evolution and maintenance by using a flexible automatic code generator. In: *Proceedings of ICSE-17 – 17th International Conference on Software Engineering*, Seattle, 1995.
- 6 *The ProgGen program generator*. 2000, October 12 [online]. – URL: www.informatics.sintef.no/projects/proggen/.
- 7 Johansen, U. *SEP sluttrapport : Designstyrt oversetter fra SDL*. Trondheim, SINTEF, 1999. (Sintef report STF40 A99040.) (ISBN 82-14-01729-7)
- 8 ITU-T. *CCITT Specification and Description Language*. Geneva, ITU-T, 1994. (Recommendation Z.100 (SDL'92).)
- 9 ITU-T. *CCITT Specification and Description Language*. Geneva, ITU-T, 1999. (Recommendation Z.100 (11/99).)
- 10 ITU-T. *SDL Methodology Guidelines*. Geneva, ITU-T, 1993. (Recommendation Z.100 (03/93) Appendix I.)
- 11 Floch, J. *Using UML for architectural design of SDL systems*. Trondheim, SINTEF, 2000. (SINTEF report STF 40 A00009.)
- 12 Mansurov, N, Probert, R L. Dynamic scenario-based approach to re-engineering of legacy telecommunication software. In: *SDL'99 The Next Millennium, Proceedings of the Ninth SDL Forum*. Amsterdam, Elsevier Science, 1999.
- 13 *The SISU project*. 2000, October 12 [online]. – URL: www.sintef.no/sisu.
- 14 Crossing the fjord. *Signals*. Malmø, Telelogic, 1999. (Telelogic Newsletter No 2.) (www.telelogic.se)
- 15 Kerbrat, A, Jéron, T, Groz, R. Automated test generation from SDL specifications. In: *SDL'99 The Next Millennium, Proceedings of the Ninth SDL Forum*. Amsterdam, Elsevier Science, 1999.
- 16 Bræk, R et al. Quality by construction exemplified by TIME – The Integrated Methodology. *Elektronikk*, 95 (1), 73–82, 1999.

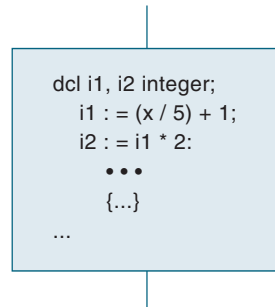


Figure 10 Textual tasks in SDL 2000